

PROGRAMMING LANGUAGES

STATICALLY TYPED IMPERATIVE LANGUAGE IN ANTLR

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.Y. 2025/2026



© Michele Pasqua, 2026

LANGUAGES AND TYPES



LANGUAGE TYPEDENESS

Every modern programming language performs type checking on programs

- ▶ Even in languages without variable (typed) declarations *// e.g., Python*
- ▶ Except single-type, or **typeless**, languages *// e.g., x86 ASM*

The point is **when** this is done

- ▶ At compile-time (statically-typed) *// e.g., Java, C/C++*
- ▶ At run-time (dynamically-typed) *// e.g., Python, JavaScript*

LANGUAGE TYPEDENESS (CONT'D)

Dynamically-typed language

```
x = 5      int
⋮
x = "5"    str
```

Flexible, but programs may crash

Statically-typed language

```
int x = 5;  int
⋮
x = "5";
```

Strict, but programs cannot crash

Typedness also concerns the **permissiveness** of type compatibility

- ▶ Flexible type coercion (weakly-typed)
- ▶ Constrained type coercion (strongly-typed)

// e.g., JavaScript, C/C++

// e.g., Java, Python

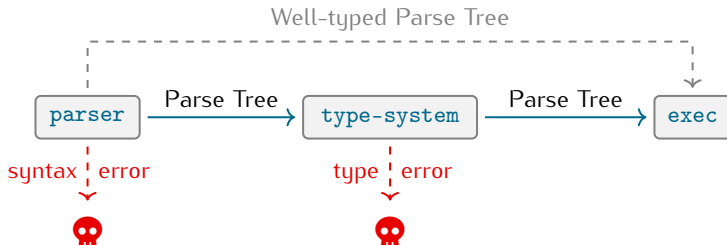
STATICALLY TYPING

Our first imperative language `Imp` is **dynamically** (but strongly) typed

We now make it statically typed

// ... as it should always be

- We need a **type system** to check type compliance at compile-time



A SIMPLE TYPE SYSTEM



STATICALLY TYPED IMPERATIVE LANGUAGE

We add variable type declarations to our imperative language TypedImp

- ▶ List of the form `string x; ...; dec y;` at program head
- ▶ Variables have **single** type declarations

We now need a **type system** for the language

- ▶ Inferring types for each Parse Tree element
- ▶ Checking type compliance with variable declarations and command semantics
- ▶ Working at compile-time (i.e., before trying to run a program)

TYPE SYSTEM

An inference process trying to assign types to program constructs

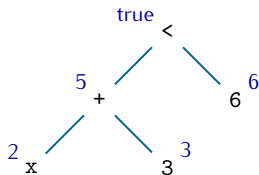
- ▶ If a program can be assigned with a (legitimate) type, it is **well-typed**
- ▶ In the simplest cases, this is a decision procedure

We will go through a more practical approach

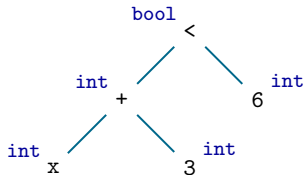
- ▶ We implement the type system as an **interpreter** for the language
- ▶ In this interpreter, computation is on **types** instead of values
- ▶ Which value should the visit methods should return?

EXPRESSION EVALUATION

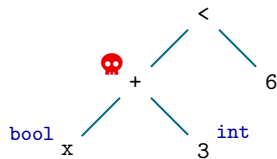
Imp



TypedImp



TypedImp



Type of expressions is used to check type compliance of commands

- No type coercion is performed

TYPE INTERPRETER SEMANTICS

Typing $\Gamma \in \mathbb{X} \rightarrow \mathbb{T}$ mapping variables to types

- Types are the ones defined in the language: `int`, `dec`, `bool`, `string`

$$\text{DECL} \frac{-}{\langle \Gamma, \tau \ x \rangle \longrightarrow \langle \Gamma[\tau/x], \epsilon \rangle} \Gamma(x) \text{ undefined}$$

$$\text{ASSIGN} \frac{\langle \Gamma, \text{exp} \rangle \longrightarrow \text{exp}'}{\langle \Gamma, x = \text{exp} \rangle \longrightarrow \langle \Gamma, x = \text{exp}' \rangle}$$

$$\text{ASSIGNE} \frac{-}{\langle \Gamma, x = \tau \rangle \longrightarrow \langle \Gamma, \epsilon \rangle} \tau = \Gamma(x)$$

TYPE INTERPRETER SEMANTICS (CONT'D)

Conditionals

$$\text{If} \frac{\langle \Gamma, \text{exp} \rangle \longrightarrow \text{exp}'}{\langle \Gamma, \text{if } (\text{exp}) \{ \text{com} \} \rangle \longrightarrow \langle \Gamma, \text{if } (\text{exp}') \{ \text{com} \} \rangle}$$

$$\text{IfE} \frac{-}{\langle \Gamma, \text{if } (\tau) \{ \text{com} \} \rangle \longrightarrow \langle \Gamma, \text{com} \rangle} \quad \tau = \text{bool}$$

What happens when `exp` does not evaluate to `bool`?

- The interpreter should raise an error, meaning the typing has failed

TYPE SYSTEM SEMANTICS (CONT'D)

Finally, sequences of commands

$$\text{SEQ} \frac{\langle \Gamma, \text{com}_1 \rangle \longrightarrow \langle \Gamma', \text{com}'_1 \rangle}{\langle \Gamma, \text{com}_1 ; \text{com}_2 \rangle \longrightarrow \langle \Gamma', \text{com}'_1 ; \text{com}_2 \rangle}$$

$$\text{SEQE} \frac{-}{\langle \Gamma, \epsilon ; \text{com}_2 \rangle \longrightarrow \langle \Gamma, \text{com}_2 \rangle}$$

Evaluation stops when `com` yields an empty configuration $\langle \Gamma, \epsilon \rangle$

- Meaning that the program is **well-typed**

ADDING FEATURES



TYPE CONVERSION

Add to the language the possibility of performing

- ▶ Implicit **type coercion**
- ▶ Explicit **type casting**

// widening conversion

// narrowing conversion

Type coercion

```
int x;  
dec y;  
x = 5;  
y = 2.3 + x; // x implicitly converted to dec  
print toStr(y) // print on screen: 7.3
```

Only **int** to **dec** allowed

Type casting

```
int x;  
dec y;  
y = 2.3;  
x = 5 + (int)y; // y explicitly converted to int  
print toStr(x) // print on screen: 7
```

Only **dec** to **int** allowed

MORE FLEXIBLE DECLARATIONS

Add the possibility of declaring variables anywhere in the program

- ▶ Declarations may (optionally) allow to initialize variables
- ▶ Variable visibility is limited by their **scope** (closest surrounding braces)
- ▶ Variable **shadowing** is not allowed, check it with the type system

```
dec x;  
x = 4.3;  
if (x < 7) {      // here z is not visible, but x is visible  
  dec y = 0.0;  
  dec x;         // not allowed, compile-time error  
  y = y + 1  
} else {         // here y is not visible, but x is visible  
  dec z = 0.0;  
  z = z - 1  
};  
print toStr(x)  // here y and z are not visible
```

Q + A



© Michele Pasqua, 2026

This work is licensed under the  **Creative Commons** Attribution-NonCommercial-NoDerivatives 4.0 International license. To view a copy of the license, visit:

<http://creativecommons.org/licenses/by-nc-nd/4.0>

You are free to:

 **Share** – Copy, distribute, and display the work

Under the following terms:

 **Attribution** – You must attribute the work in the manner specified by the author or licensor

 **NonCommercial** – You may not use this work for commercial purposes

 **NoDerivatives** – If you alter, transform, or build upon this work, you may not distribute the modified material

For any reuse or distribution, you must make clear to others the license terms of this work. Any of these conditions can be waived if you get permission from the copyright holder. Your fair use and other rights are in no way affected by the above.